Immutability & Persistent Collections

ersistent Collections · Immutable playlist edits sharing structure · Build with Waqa

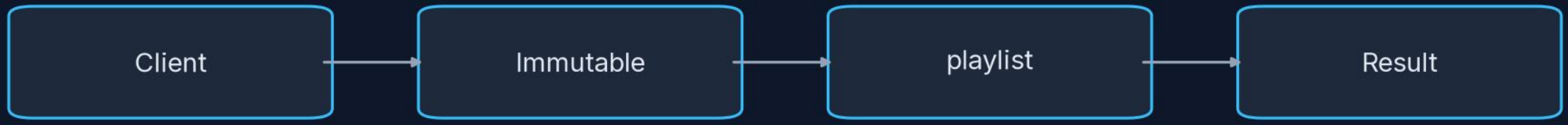
Today's Hook

- Problem: Immutable playlist edits sharing structure
- Pattern: Persistent Collections — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Immutable playlist edits sharing structure
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Persistent Collections: Immutable playlist edits sharing structure

Real-World Scenario

- Running example: Immutable playlist edits sharing structure
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.functional.collections.problem;

import java.util.*;

public final class NaivePlaylist {
  public final List<String> songs=new ArrayList<>();
  public void add(String s){ songs.add(s); }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.functional.collections.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

final class Playlist {
  private final List<String> songs;
  Playlist(List<String> songs){ this.songs=List.copyOf(songs); }
  Playlist add(String s){ var n=new ArrayList<>(songs); n.add(s); return new Playlist(n); }
  List<String> songs(){ return songs; }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.collections.problem;
import java.util.*; import java.util.concurrent.*; import java.util.concurrent.atomic.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Persistent collections: playlist edits ===");
    System.out.println("--- naive ---"); var n=new NaivePlaylist(); n.add("a"); System.out.println(n.songs);
System.out.println("--- persistent ---");
var p=new Playlist(List.of("a")); var p2=p.add("b");
System.out.println(p.songs()); System.out.println(p2.songs());
  }
}
```

Walkthrough Output

```
=== Persistent collections: playlist edits ===
--- naive ---
[a]
--- persistent ---
[a]
[a, b]
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Persistent Collections.

Related Patterns

- Contrast Persistent Collections with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Persistent Collections → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.